

Báo cáo bài tập số 4

Họ và tên: Nguyễn Thành Nam

MSSV: 20225368

1. Mã nguồn:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/socket.h>

#include <arpa/inet.h>

#include <netdb.h>


#define MAX_BUF 2048

#define FTP_SERVER "lebavui.io.vn"

#define FTP_PORT 21


void send_cmd(int sock, const char *cmd) {

    send(sock, cmd, strlen(cmd), 0);

    printf("-> %s", cmd);

}


void recv_resp(int sock, char *buffer) {

    memset(buffer, 0, MAX_BUF);

    recv(sock, buffer, MAX_BUF - 1, 0);
```

```
printf("<- %s", buffer);  
}
```

```
int connect_to_server(const char *hostname, int port) {
```

```
    int sock;
```

```
    struct sockaddr_in server_addr;
```

```
    struct hostent *host;
```

```
    if ((host = gethostbyname(hostname)) == NULL) {
```

```
        perror("[-] gethostbyname lỗi");
```

```
        return -1;
```

```
    }
```

```
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
```

```
        perror("[-] Lỗi tạo socket");
```

```
        return -1;
```

```
    }
```

```
    server_addr.sin_family = AF_INET;
```

```
    server_addr.sin_port = htons(port);
```

```
    server_addr.sin_addr = *((struct in_addr *)host->h_addr);
```

```
    memset(&(server_addr.sin_zero), 0, 8);
```

```
    if (connect(sock, (struct sockaddr *)&server_addr, sizeof(struct sockaddr)) < 0) {
```

```
        perror("[-] Lỗi kết nối");
```

```
        return -1;
```

```

    }
    return sock;
}

// Hàm lấy IP và Port cho kênh dữ liệu từ phản hồi PASV
int open_pasv_data_conn(int ctrl_sock, char *buffer) {
    send_cmd(ctrl_sock, "PASV\r\n");
    recv_resp(ctrl_sock, buffer);

    int h1, h2, h3, h4, p1, p2;
    // Bóc tách IP và Port từ chuỗi: "227 Entering Passive Mode (h1,h2,h3,h4,p1,p2)"
    char *start = strchr(buffer, '(');
    if (start != NULL) {
        sscanf(start, "(%d,%d,%d,%d,%d,%d)", &h1, &h2, &h3, &h4, &p1, &p2);
    } else {
        printf("[-] Không thể parse phản hồi PASV.\n");
        return -1;
    }

    char data_ip[32];
    snprintf(data_ip, sizeof(data_ip), "%d.%d.%d.%d", h1, h2, h3, h4);
    int data_port = p1 * 256 + p2;

    return connect_to_server(data_ip, data_port);
}

```

```

void reverse_string(char *str, int len) {
    for (int i = 0; i < len / 2; i++) {
        char temp = str[i];
        str[i] = str[len - 1 - i];
        str[len - 1 - i] = temp;
    }
}

```

```

int main() {
    const char *username = "user_20225368";
    const char *password = "536812";
    char cmd[100], buffer[MAX_BUF];

    printf("=== BÀI TẬP 04.02 - FTP CLIENT (C Sockets) ===\n");
    printf("[*] Đang kết nối tới %s...\n", FTP_SERVER);

    int ctrl_sock = connect_to_server(FTP_SERVER, FTP_PORT);
    if (ctrl_sock < 0) return 1;

    recv_resp(ctrl_sock, buffer);

    snprintf(cmd, sizeof(cmd), "USER %s\r\n", username);
    send_cmd(ctrl_sock, cmd);
    recv_resp(ctrl_sock, buffer);

    snprintf(cmd, sizeof(cmd), "PASS %s\r\n", password);

```

```

send_cmd(ctrl_sock, cmd);
recv_resp(ctrl_sock, buffer);

if (strncmp(buffer, "230", 3) != 0) {
    printf("[-] Đăng nhập thất bại. Vui lòng kiểm tra lại tài khoản trên server.\n");
    close(ctrl_sock);
    return 1;
}

printf("[+] Đăng nhập thành công với tài khoản: %s!\n\n", username);

// 2. Lấy danh sách file (NLST)
int data_sock = open_pasv_data_conn(ctrl_sock, buffer);
send_cmd(ctrl_sock, "NLST\r\n");
recv_resp(ctrl_sock, buffer); // Nhận 150 File status okay

char file_list[MAX_BUF] = {0};
int bytes_read;
while ((bytes_read = recv(data_sock, buffer, MAX_BUF - 1, 0)) > 0) {
    buffer[bytes_read] = '\0';
    strcat(file_list, buffer);
}

close(data_sock);
recv_resp(ctrl_sock, buffer); // Nhận 226 Transfer complete

// Tìm file question_XXXXXX.txt
char question_file[50] = {0};

```

```

char *line = strtok(file_list, "\r\n");
while (line != NULL) {
    if (strncmp(line, "question_", 9) == 0 && strstr(line, ".txt") != NULL) {
        strncpy(question_file, line, sizeof(question_file));
        break;
    }
    line = strtok(NULL, "\r\n");
}

```

```

if (strlen(question_file) == 0) {
    printf("[-] Không tìm thấy file question_XXXXXX.txt trên server.\n");
    close(ctrl_sock);
    return 1;
}

printf("[*] Đã tìm thấy file đề bài: %s\n\n", question_file);

```

// 3. Tải file (RETR)

```

data_sock = open_pasv_data_conn(ctrl_sock, buffer);
snprintf(cmd, sizeof(cmd), "RETR %s\r\n", question_file);
send_cmd(ctrl_sock, cmd);
recv_resp(ctrl_sock, buffer); // Nhận 150

char file_content[MAX_BUF] = {0};
bytes_read = recv(data_sock, file_content, MAX_BUF - 1, 0);
close(data_sock);
recv_resp(ctrl_sock, buffer); // Nhận 226

```

```

if (bytes_read > 0) {
    file_content[bytes_read] = '\0';
}

printf("[*] Nội dung file gốc: %s\n", file_content);

// 4. Xử lý đảo ngược chuỗi và tạo file answer
char answer_file[50];
strncpy(answer_file, question_file, sizeof(answer_file));
strncpy(answer_file, "answer", 6); // Đổi "question" thành "answer"

int content_len = strlen(file_content);

// Xóa ký tự newline ở cuối nếu có để đảo ngược cho chuẩn xác
while (content_len > 0 && (file_content[content_len-1] == '\r' ||
file_content[content_len-1] == '\n')) {
    file_content[content_len-1] = '\0';
    content_len--;
}

reverse_string(file_content, content_len);

printf("[*] File trả lời: %s\n", answer_file);
printf("[*] Nội dung sau khi đảo ngược: %s\n\n", file_content);

// 5. Upload file trả lời (STOR)
data_sock = open_pasv_data_conn(ctrl_sock, buffer);
snprintf(cmd, sizeof(cmd), "STOR %s\r\n", answer_file);

```

```

send_cmd(ctrl_sock, cmd);

recv_resp(ctrl_sock, buffer); // Nhận 150

send(data_sock, file_content, content_len, 0);

close(data_sock);

recv_resp(ctrl_sock, buffer); // Nhận 226

printf("[+] Đã upload thành công file %s lên server.\n\n", answer_file);

// Thoát

send_cmd(ctrl_sock, "QUIT\r\n");

recv_resp(ctrl_sock, buffer);

close(ctrl_sock);

printf("[*] Hoàn thành chương trình!\n");

return 0;

}

```

Giải thích mã nguồn:

Mô hình FTP hoạt động dựa trên hai kênh riêng biệt: **kênh điều khiển** (Control Channel) dùng để gửi lệnh/nhận trạng thái và **kênh dữ liệu** (Data Channel) dùng để truyền tải file. Chương trình C Sockets được thiết kế để xử lý linh hoạt cả hai kênh này qua các bước sau:

- **Thiết lập kết nối và Đăng nhập:** * Hàm `connect_to_server()` tạo một TCP socket kết nối tới server `lebavui.io.vn` tại port 21 (kênh điều khiển).
 - Chương trình tự động gán tài khoản `user_20225368` và mật khẩu `536812` (theo quy tắc MSSV và ngày sinh), sau đó gửi lần lượt lệnh `USER` và `PASS` qua kênh điều khiển để xác thực.
- **Chế độ Bị động (Passive Mode) và Truyền dữ liệu:**

- Mỗi khi cần lấy danh sách file, tải file hoặc up file, giao thức FTP yêu cầu mở một luồng dữ liệu riêng. Hàm `open_pasv_data_conn()` sẽ gửi lệnh PASV tới server.
- Server trả về một chuỗi chứa IP và Port động (ví dụ: 227 Entering Passive Mode (h1,h2,h3,h4,p1,p2)). Hàm này bóc tách toán học để tìm ra Port dữ liệu ($p1 * 256 + p2$), sau đó tạo một socket thứ hai kết nối tới IP/Port này.
- **Luồng xử lý bài tập chính:**
 - **Lấy danh sách (NLST):** Mở kênh PASV, gửi lệnh NLST. Chương trình nhận chuỗi văn bản chứa danh sách file, dùng hàm `strtok` tách từng dòng và lọc ra tên file có định dạng `question_XXXXXX.txt`.
 - **Tải file (RETR):** Mở kênh PASV mới, gửi lệnh RETR kèm tên file vừa tìm được. Nội dung 100 ký tự được đọc từ socket và lưu vào bộ nhớ đệm (buffer).
 - **Xử lý đảo ngược:** Cắt bỏ các ký tự xuống dòng (nếu có), dùng hàm `reverse_string()` để đổi chỗ các phần tử trong mảng (phần tử đầu đổi cho phần tử cuối) nhằm đảo ngược hoàn toàn chuỗi. Tên file cũng được thay thế chuỗi "question" bằng "answer".
 - **Upload (STOR):** Mở kênh PASV cuối cùng, gửi lệnh STOR kèm tên file mới, sau đó đẩy chuỗi đã đảo ngược qua kênh dữ liệu để nộp bài. Cuối cùng, gửi QUIT để đóng phiên làm việc một cách an toàn.

2. Kết quả:

- **[*] Nội dung sau khi đảo ngược:** Hiển thị chuỗi kết quả để giảng viên đối chiếu tính đúng đắn của thuật toán lập trình trong C.
- **[+] Đã upload thành công...** Là lời khẳng định quá trình gửi file answer_XXXXXX.txt lên server qua lệnh STOR không gặp lỗi kết nối hay phân quyền.